# Alpha Z

Technical Assessment

## Background

Read the CORAL paper **(CORAL: Towards Autonomous Multi-Agent Evolution for Open-Ended Discovery, arXiv 2604.01658)** and its open-source repository at https://github.com/Human-Agent-Society/CORAL, including the paper, code, and example tasks.

## 1 Run CORAL and Replicate at Least One Task Result

Deploy CORAL locally or in the cloud. Choose any task from the repository examples/ directory and run it using any available model (open-source models such as Qwen or DeepSeek are acceptable).

**Report your experimental results and compare with the paper:**

- Final Score
- Number of evaluations (#Evals)
- Comparison table against the paper numbers

## 2 Apply CORAL to an OR Problem

Set up a complete CORAL task for a classic combinatorial optimization problem of your choice (TSP, scheduling, bin packing, VRP, etc.). You must write:

- **grader.py —** evaluator that checks constraints and computes solution quality
- **Seed code —** a valid initial solution
- **task.yaml —** task configuration

Run the experiment and report solution quality: how much does CORAL improve over the naive seed or a known baseline?

## 3 Ablation Study: Which CORAL Mechanism Matters for Your OR Problem

Using the same task from Task 2, run the following two ablation conditions. Each condition must be

run at least 3 times. Report mean ± std of Final Score.

- **Condition A —** disable notes/skills (remove knowledge accumulation) vs. full CORAL
- **Condition B —** disable all heartbeats vs. full CORAL

Provide the numerical results and describe what you observe.

## 4 Improve CORAL: Knowledge Distillation and Memory Management

CORAL accumulates notes and skills continuously across evaluations, but has no mechanism to forget, consolidate, or prioritize knowledge. In long-running experiments, this leads to three observable problems:

- **Noise accumulation**: low-quality or contradictory notes are never discarded
- **Retrieval degradation**: as the knowledge base grows, relevant skills become harder to surface
- **Redundancy**: similar discoveries are stored multiple times without consolidation

Your task is to design and implement an improvement to CORAL's knowledge accumulation mechanism.

**You must:**

- Identify at least one concrete failure mode in CORAL's current notes/skills system (show evidence from your Task 2 or Task 3 runs)
- Propose and implement a modification to address it. Example directions include structured memory compression, quality-filtered writing, or periodic distillation — but you are free to choose your own approach
- Run a comparative experiment: your modified CORAL vs. vanilla CORAL on the same OR task from Task 2. Report mean ± std of Final Score over at least 3 runs per condition

**The modification must be non-trivial:** changing a prompt or a hyperparameter does not qualify. You must modify or extend CORAL's codebase.

## 5 Product Plan: LLM-Driven Autonomous Combinatorial Optimization Tool

Assume you are responsible for developing an LLM-driven autonomous optimization tool for enterprise customers (scheduling, routing, or resource allocation). The tool is based on CORAL and any other references you choose.

Provide a concrete technical plan covering all five of the following points:

① **System Architecture**
Overall module breakdown and data flow.

② **Knowledge Accumulation Mechanism**
How solving history, user cases, and domain experience are accumulated across tenants and reused.

③ **Multi-Tenant Isolation Strategy**
How data, models, and solving history are isolated between enterprise customers while still sharing general knowledge.

④ **Fuzzy User Feedback Driving Self-Evolution**
CORAL relies on a structured evaluator that returns a precise numeric score. Enterprise users, however, give vague feedback such as "this schedule doesn't work" or "the drivers are too tired". How do you convert this kind of feedback into a signal that can drive agent evolution?

⑤ **Foreseeable Failure Modes**
List where you expect this type of system to break down most often, and the corresponding mitigation strategies.

## 6 Anything Else You Want to Add

Any other projects, ideas, or experience in operations research or AI for OR are welcome.

## Submission

**GitHub repository + brief readme.**
Tasks 1–4 are evaluated on experimental results and runnable code. Task 5 is evaluated on technical judgement. Written prose quality is not assessed.